David Weather Predictor
Execution Guide

David H. Haindongo

February 26, 2026

Contents

1 System Overview **3**
1.1 High-Level Architecture . . . 3

2 Execution Pipeline – Step by Step **3**
2.1 Step 1: Data Ingestion . . . 3
2.2 Step 2: Feature Engineering . . . 4
2.2.1 Lags . . . 4
2.2.2 Rolling Statistics . . . 4
2.2.3 Rate of Change and Acceleration . . . 5
2.2.4 Rain Accumulation . . . 5
2.2.5 Interaction Terms . . . 6
2.2.6 Fast Fourier Transform (FFT) Spectral Features . . . 6
2.2.7 Rolling Correlations . . . 6
2.2.8 Cyclical Time Features . . . 6
2.2.9 Principal Component Analysis (PCA) . . . 7
2.3 Step 3: Hidden Markov Model (HMM) Regime Detection . . . 7
2.4 Step 4: Target Creation . . . 7
2.5 Step 5: Training the Stacked Ensemble . . . 8
2.5.1 5.1 Data Splitting . . . 8
2.5.2 5.2 Feature Selection . . . 8
2.5.3 5.3 Scaling . . . 8
2.5.4 5.4 Synthetic Minority Over-sampling Technique (SMOTE) . . . 8
2.5.5 5.5 Hyperparameter Tuning for Each Base Model . . . 8
2.5.6 5.6 Calibration with Platt Scaling . . . 9
2.5.7 5.7 Per-Model Optimal Thresholds . . . 9
2.5.8 5.8 Meta-Features and Meta-Learner . . . 9
2.5.9 5.9 Final Evaluation on Test Set . . . 9
2.5.10 5.10 Conformal Quantile . . . 9
2.6 Step 6: Generating a Prediction for the Latest Time Step . . . 9
2.7 Step 7: Trust Assessment . . . 10

3 Key Definitions and Terms **10**

4 Important Functions and Their Roles **11**

5 Examples of Input/Output **12**

6 Conclusion **12**

1 System Overview

The David Weather Predictor is a machine learning pipeline that ingests historical and real-time weather data, engineers a rich set of features, detects atmospheric regimes, and trains a stacked ensemble of eight diverse models. The ensemble is calibrated, optimised for the F2 score (a metric that weights recall twice as much as precision), and provides prediction intervals via conformal prediction. A trust assessment module helps users interpret the reliability of each forecast.

1.1 High-Level Architecture

The main components and data flow are:

```
Data Ingestion
|
v
Feature Engineering
|
v
Hidden Markov Model (HMM) Regime Detection
|
v
Target Creation
|
v
Training Pipeline
|
v
Prediction
|
v
Trust Assessment
```

2 Execution Pipeline – Step by Step

2.1 Step 1: Data Ingestion

Purpose: Acquire live and historical weather data from the Open-Meteo Application Programming Interface (API), handle missing values, and compute a data quality score.

Why Important: Reliable forecasts depend on clean, complete data. Missing values are common in real-world datasets; proper handling prevents bias and information loss.

Process:

1. Fetch up to 92 days of hourly data (Open-Meteo free tier limit).
2. If API fails, generate 1 year of synthetic data with realistic seasonal and diurnal cycles.

3. Apply time-based linear interpolation for short gaps.
4. Use K-Nearest Neighbours (KNN) imputation for remaining missing values ($k = 5$, distance-weighted).
5. Fill any remaining gaps with column median.
6. Compute data quality = 1 – (missing cells / total cells).

Key Functions: `fetch_from_openmeteo()`, `handle_missing_data()`, `_create_enhanced_synthe`

Example: Input: DataFrame with 25.7% missing cells. Output: Fully imputed DataFrame, data quality = 100%.

2.2 Step 2: Feature Engineering

Purpose: Transform raw meteorological variables into a rich set of predictors that capture temporal patterns, interactions, and cyclic behaviours.

Why Important: Raw variables alone (temperature, humidity, pressure) are insufficient for accurate rainfall prediction. Engineered features expose lag effects, trends, volatility, and periodicities that models can leverage.

Below is a detailed explanation of each generated feature type, including its definition, purpose, and a concrete example.

2.2.1 Lags

Definition: A lagged variable is the value of a meteorological variable (e.g., temperature) from a previous time step. For a given time t, the lag h is the value at time $t - h$.

Purpose: Lags capture the recent history of the atmosphere. Weather systems often persist, so past values are strong predictors of future conditions.

Example: If temperature at 10:00 is 20°C, then the 3-hour lag at 13:00 is 20°C. The feature `temperature_lag_3h` at 13:00 would be 20.

2.2.2 Rolling Statistics

Definition: Rolling statistics are summary measures computed over a moving window of fixed length. We compute:

- **Mean:** average over the window
- **Standard deviation:** measure of variability
- **Minimum:** smallest value
- **Maximum:** largest value
- **Skewness:** asymmetry of the distribution
- **Kurtosis:** "tailedness" of the distribution

Purpose: These features summarise the local behaviour of a variable. For example, the rolling mean smooths short-term fluctuations, while the rolling standard deviation captures volatility (e.g., approaching storms often increase pressure variability).

Example: Over a 6-hour window, pressure readings are [1013, 1015, 1014, 1013, 1012, 1010]. Then:

$$\begin{aligned} \text{mean} &= \frac{1013 + 1015 + 1014 + 1013 + 1012 + 1010}{6} = 1012.83 \\ \text{std} &\approx 1.72 \\ \text{min} &= 1010 \\ \text{max} &= 1015 \end{aligned}$$

2.2.3 Rate of Change and Acceleration

Definition:

- **First difference** (rate of change): $\Delta_h v(t) = v(t) - v(t-h)$.
- **Second difference** (acceleration): $\Delta_h^2 v(t) = \Delta_h(\Delta_h v(t)) = v(t) - 2v(t-h) + v(t-2h)$.

Purpose: Rate of change tells how quickly a variable is increasing or decreasing (e.g., pressure drop indicates a storm). Acceleration tells whether that change is speeding up or slowing down.

Example: Pressure at $t = 0$: 1013, $t = 6$: 1015, $t = 12$: 1010.

$$\Delta_6 v(6) = 1015 - 1013 = 2, \quad \Delta_6^2 v(12) = 1010 - 2 \cdot 1015 + 1013 = -7.$$

A positive rate of change means pressure is rising; a negative acceleration means the rise is slowing down.

2.2.4 Rain Accumulation

Definition: For a given window length, we compute:

- **Sum:** total precipitation over the window.
- **Maximum:** highest hourly precipitation.
- **Standard deviation:** variability of precipitation.

Purpose: These features capture recent wetness, which is a direct precursor to future rain.

Example: Over a 6-hour window, precipitation (mm) = [0.0, 0.2, 0.5, 0.3, 0.0, 0.0]. Then:

$$\begin{aligned} \text{sum} &= 1.0 \text{ mm}, \\ \text{max} &= 0.5 \text{ mm}, \\ \text{std} &\approx 0.22. \end{aligned}$$

2.2.5 Interaction Terms

Definition: Products of two variables, often scaled to avoid numerical issues. We use:

- `temp_humidity` = (temperature × humidity) / 100.
- `pressure_humidity` = (pressure × humidity) / 1000.

Purpose: Interactions capture combined effects. For example, high humidity alone may not cause rain, but high humidity combined with low pressure is a stronger signal.

Example: If temperature = 20°C, humidity = 80%, then

$$\text{temp_humidity} = \frac{20 \times 80}{100} = 16.$$

2.2.6 Fast Fourier Transform (FFT) Spectral Features

Definition: The Fast Fourier Transform decomposes a time series into its frequency components. For a 168-hour (1 week) window, we extract:

- **Top 5 dominant frequencies:** the frequencies with the largest amplitudes.
- **Total spectral energy:** sum of squared amplitudes.

Purpose: These features capture periodic patterns, such as diurnal cycles (24h), synoptic waves (3-7 days), or longer oscillations.

Example: A pressure series might show a dominant frequency of 0.0417 cycles/hour (24-hour period) and a total energy of 450.

2.2.7 Rolling Correlations

Definition: Pearson correlation coefficient between two variables over a rolling 72-hour window:

$$\rho_{xy}(t) = \frac{\sum_{i=t-71}^{t}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum(x_i - \bar{x})^2 \sum(y_i - \bar{y})^2}}.$$

We compute correlations for (pressure, temperature), (pressure, humidity), and (temperature, humidity).

Purpose: Relationships between variables can change with weather regimes. For example, during a storm, pressure and humidity might become strongly anti-correlated.

Example: If pressure and temperature have been rising together over the last 72 hours, the correlation will be close to +1.

2.2.8 Cyclical Time Features

Definition: We encode time using both linear and trigonometric transformations:

- `hour`, `day_of_year`, `month` – linear representations.
- Sine/cosine transforms: `hour_sin`, `hour_cos` (daily cycle), `doy_sin`, `doy_cos` (annual cycle), `week_sin`, `week_cos` (weekly cycle).

Purpose: Cyclical features allow models to learn periodic patterns without assuming that hour 23 is close to hour 0. The sine/cosine pair preserves the circular nature of time.

Example: For hour = 0, `hour_sin`=0, `hour_cos`=1. For hour = 6, `hour_sin`=1, `hour_cos`=0.

2.2.9 Principal Component Analysis (PCA)

Definition: Principal Component Analysis is a dimensionality reduction technique that transforms a set of possibly correlated variables into a smaller number of uncorrelated variables called principal components. The first principal component captures the direction of maximum variance, the second the next largest variance orthogonal to the first, and so on.

Purpose: PCA summarises the information from hundreds of features into a handful of components, reducing noise and multicollinearity. These components often represent "latent" weather patterns.

Example: From 200 numeric features, the first principal component might be a linear combination like $0.3 \cdot \text{pressure} + 0.4 \cdot \text{temperature} - 0.2 \cdot \text{humidity} + \cdots$, and its value for a given hour could be 3.2 (unitless). We add the first five such components as features.

2.3 Step 3: Hidden Markov Model (HMM) Regime Detection

Purpose: Identify hidden atmospheric states (regimes) using a 3-state Gaussian Hidden Markov Model on standardised pressure, temperature, and humidity.

Why Important: Weather often exhibits distinct regimes (e.g., stable, transition, stormy). Knowing the current regime provides contextual information that can improve predictions.

Process:

1. Standardise the three variables.
2. Fit a 3-state Gaussian HMM (diagonal covariance).
3. Compute posterior probabilities of each state at every time step.
4. Add these probabilities as features `regime_0`, `regime_1`, `regime_2`, and also `volatile_prob` (probability of the state with highest variance).

Key Functions: `apply_hmm()`.

Example: Output for the latest hour: `regime_0`=0.98, `regime_1`=0.02, `regime_2`=0.00, `volatile_prob`=0.00 (meaning stable regime).

2.4 Step 4: Target Creation

Purpose: Define the binary prediction target: whether total precipitation over the next H hours exceeds 0.2 millimetres (mm).

Why Important: This transforms the forecasting problem into a supervised classification task.

Equation:

$$y_t = \mathbf{1}\left(\sum_{i=1}^{H} \text{precip}_{t+i} > 0.2\right)$$

Example: For $H = 4$, if the sum of precipitation in the next 4 hours is 0.5 mm, $y_t = 1$ (rain).

Key Functions: `create_target_variable()`.

2.5 Step 5: Training the Stacked Ensemble

This is the most complex stage, involving multiple sub-steps.

2.5.1 5.1 Data Splitting

The data is split chronologically into:

- Training: first 60% (used for model training after SMOTE)
- Validation: next 20% (used for calibration, threshold optimisation, conformal quantile)
- Test: last 20% (held out for final performance evaluation)

2.5.2 5.2 Feature Selection

Mutual information between each feature and the target is computed on the training set. The top 50–150 features (30% of total, capped) are retained. This reduces dimensionality and noise.

2.5.3 5.3 Scaling

All features are scaled using `RobustScaler` (median and interquartile range) to make them robust to outliers.

2.5.4 5.4 Synthetic Minority Over-sampling Technique (SMOTE)

Because rain events are often rare (class imbalance), SMOTE generates synthetic samples of the minority class (rain) by interpolating between existing minority instances. This balances the training set to 50% rain, 50% no rain.

Equation:

$$\mathbf{x}_{\text{new}} = \mathbf{x}_i + \lambda(\mathbf{x}_j - \mathbf{x}_i), \quad \lambda \in [0, 1]$$

2.5.5 5.5 Hyperparameter Tuning for Each Base Model

Eight base models are used:

1. Random Forest (RF)
2. Extreme Gradient Boosting (XGBoost)
3. Light Gradient Boosting Machine (LightGBM)
4. Logistic Regression (LR)
5. Support Vector Machine (SVM, with Radial Basis Function kernel, probability=True)
6. Gradient Boosting Trees (GBT, from scikit-learn)
7. Extra Trees (ET)
8. Multi-layer Perceptron (MLP)

For each, a randomized search over a predefined hyperparameter grid is performed using 3-fold time-series cross-validation on the SMOTE-balanced training set. The scoring metric is **F2 score** (defined below), which weights recall twice as much as precision – important because missing rain is costlier than a false alarm.

F2 score:

$$F_2 = 5\frac{P \cdot R}{4P + R}, \quad P = \frac{TP}{TP + FP}, \; R = \frac{TP}{TP + FN}$$

where TP = True Positives, FP = False Positives, FN = False Negatives.

2.5.6 5.6 Calibration with Platt Scaling

Raw probabilities from each tuned model are often poorly calibrated. A logistic regression (Platt scaling) is fitted on the validation set to map raw probabilities to well-calibrated ones.

Equation:

$$p_{\text{cal}} = \frac{1}{1 + e^{-(a\, p_{\text{raw}} + b)}}$$

where a, b are learned.

2.5.7 5.7 Per-Model Optimal Thresholds

For each calibrated model, the threshold on the validation set that maximises the F2 score is stored. This threshold is later used to convert probabilities into binary predictions when displaying individual model outputs.

2.5.8 5.8 Meta-Features and Meta-Learner

The calibrated validation probabilities from all eight models are stacked into a matrix (meta-features). A Random Forest meta-learner is trained on these meta-features and the true validation labels. This ensemble learns to combine the base models' outputs optimally.

2.5.9 5.9 Final Evaluation on Test Set

The entire pipeline (base models, calibrators, meta-learner) is applied to the test set. Performance metrics (accuracy, precision, recall, F1, F2, Area Under the Curve (AUC), confusion matrix) are computed for each base model and the final ensemble. The optimal ensemble threshold (also found on validation) is used for binary decisions.

2.5.10 5.10 Conformal Quantile

The absolute errors $|p_{\text{ens}} - y|$ on the validation set are computed. The 80th percentile q is stored as the conformal quantile. For a new prediction, the 80% prediction interval is $[p_{\text{ens}} - q,\; p_{\text{ens}} + q]$ (clipped to $[0, 1]$). This provides an uncertainty estimate.

2.6 Step 6: Generating a Prediction for the Latest Time Step

Given the current weather (the last row of the data), the pipeline produces:

- Calibrated probabilities from each base model.

- Meta-features (these probabilities) fed into the meta-learner to obtain the ensemble probability p_{ens}.
- Conformal interval: $[p_{\text{ens}} - q,\ p_{\text{ens}} + q]$.
- Final outcome: `RAIN` if $p_{\text{ens}} \geq$ ensemble threshold, else `CLEAR`.
- Confidence level based on interval width.

2.7 Step 7: Trust Assessment

A trust score is computed using a weighted combination of:

- Data quality (0–3)
- Interval width (0–2)
- Model agreement (range of base model probabilities) (0–2)
- Confidence label (0–1)
- Low probability bonus (1 if clear and $p_{\text{ens}} < 0.2$)
- Volatility penalty (subtract 0.5 if high)

The total is converted to a percentage of the maximum possible, and a trust level (HIGH, MODERATE, LOW, VERY LOW) is assigned.

Example: For a 2-hour forecast with data quality=100%, interval width=5%, model agreement range=4.1%, confidence=VERY HIGH, low probability bonus, no volatility, trust score=100% (HIGH).

3 Key Definitions and Terms

SMOTE Synthetic Minority Over-sampling Technique: creates synthetic samples of the minority class by interpolating between existing ones.

F2 Score A metric that weights recall twice as much as precision: $F_2 = 5\frac{P \cdot R}{4P+R}$, where P is precision and R is recall.

Platt Scaling A calibration method that fits a logistic regression to map raw probabilities to calibrated ones.

Conformal Prediction A method to produce prediction intervals with guaranteed coverage (here 80%) by using the quantile of absolute errors on a validation set.

Meta-Learner A model (here Random Forest) trained on the outputs of base models to combine them.

HMM Hidden Markov Model: a statistical model where the system is assumed to be a Markov process with unobserved (hidden) states.

PCA Principal Component Analysis: a dimensionality reduction technique that creates uncorrelated linear combinations of the original features explaining maximum variance.

FFT Fast Fourier Transform: an algorithm to compute the discrete Fourier transform, revealing periodic components in a signal.

AUC Area Under the Receiver Operating Characteristic Curve: a measure of a model's ability to distinguish between classes.

API Application Programming Interface: a set of protocols for building and interacting with software applications.

KNN K-Nearest Neighbours: an algorithm that uses the k closest data points to estimate missing values or make predictions.

XGBoost Extreme Gradient Boosting: an optimized gradient boosting library designed for speed and performance.

LightGBM Light Gradient Boosting Machine: a gradient boosting framework that uses tree-based learning and is optimized for efficiency.

RF Random Forest: an ensemble learning method that constructs multiple decision trees and outputs their average prediction.

LR Logistic Regression: a linear model for binary classification that outputs probabilities using a logistic function.

SVM Support Vector Machine: a classifier that finds the hyperplane that best separates classes, with extensions for probability outputs.

GBT Gradient Boosting Trees: an ensemble technique that builds trees sequentially, each correcting errors of the previous one.

ET Extra Trees: an ensemble method similar to Random Forest but with random splits, often providing better variance reduction.

MLP Multi-layer Perceptron: a class of feedforward artificial neural networks.

4 Important Functions and Their Roles

- `ingest_data()`: Orchestrates data fetching and missing value handling.
- `engineer_features_expert()`: Creates all advanced features.
- `apply_hmm()`: Fits the Hidden Markov Model and adds regime probabilities.
- `create_target_variable()`: Generates binary labels for a given horizon.
- `train_optimized_ensemble()`: Executes the entire training pipeline (feature selection, SMOTE, tuning, calibration, meta-learning).
- `predict()`: Produces a forecast for the current conditions.
- `assess_trust()`: Computes trust score and reasons.
- `save_results()`: Saves predictions to '.csv' and '.xlsx' files.

5 Examples of Input/Output

Input (at prediction time)

```
Latest weather: temperature=14.5\textdegree C, humidity=78%, pressure=834.6 hPa,
cloud_cover=96%, precipitation=0.0 mm, wind_speed=..., wind_direction=...
```

Output (for 2-hour forecast)

```
Individual Model Predictions:
RF: 5.7% (thresh=0.07) -> CLEAR
XGB: 3.9% (thresh=0.04) -> CLEAR
LGBM: 3.1% (thresh=0.08) -> CLEAR
LR: 2.8% (thresh=0.06) -> CLEAR
SVM: 1.8% (thresh=0.09) -> CLEAR
GBT: 5.9% (thresh=0.06) -> CLEAR
ET: 4.0% (thresh=0.10) -> CLEAR
MLP: 4.2% (thresh=0.05) -> CLEAR

Stacked Ensemble:
Probability: 8.0%
80% CI: [5.5%, 10.5%]
Interval width: 5.0%
Ensemble threshold: 0.36
Final outcome: CLEAR SKIES
Confidence: VERY HIGH
Trust Level: HIGH
```

6 Conclusion

The David Weather Predictor is a sophisticated system that combines domain-inspired feature engineering, state-of-the-art machine learning, and rigorous probabilistic methods to deliver accurate and trustworthy forecasts. Its modular design allows easy adaptation to other locations and time horizons.